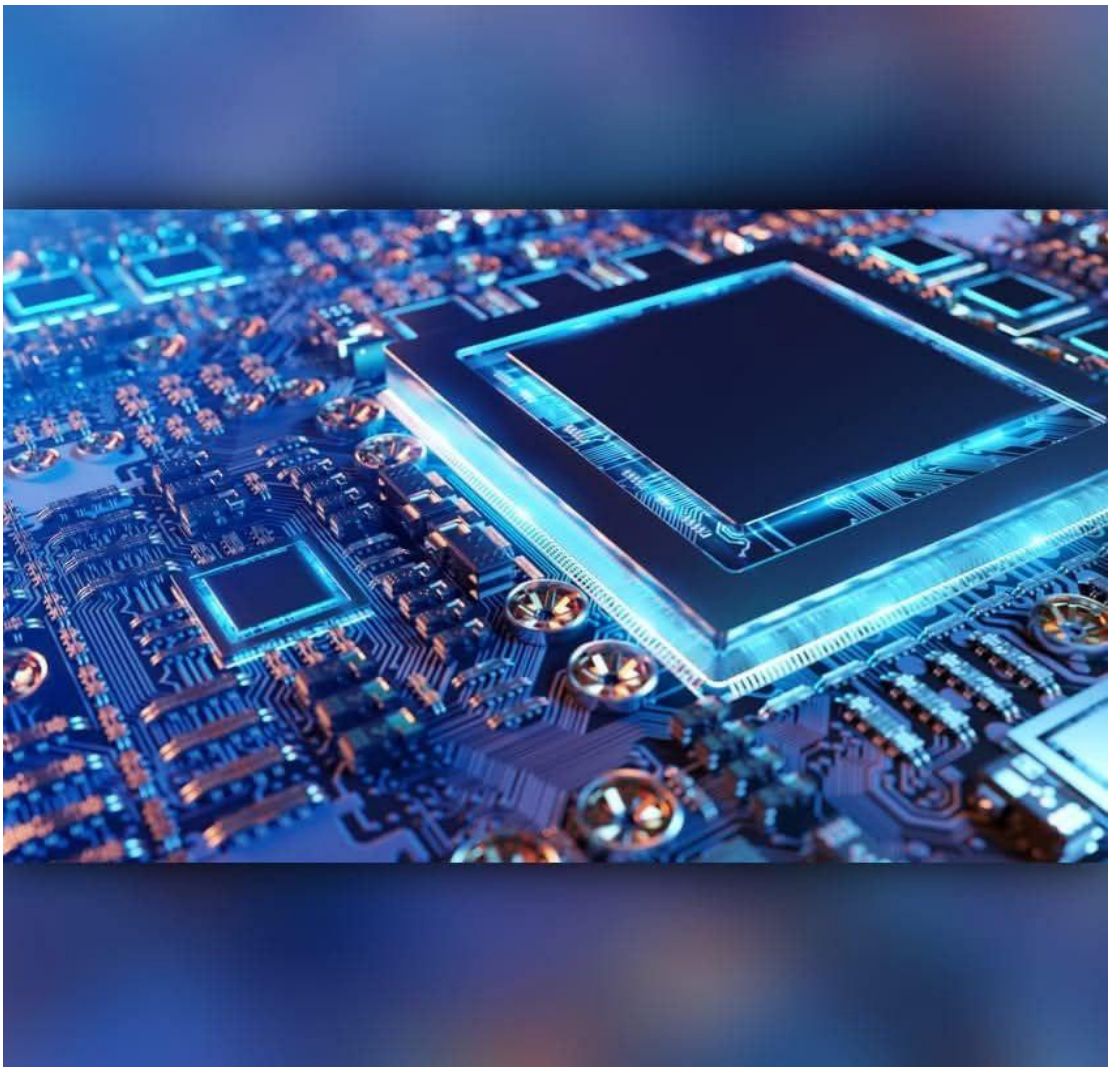
Digital Electronics Design
Assignment_Three

Prepared by:

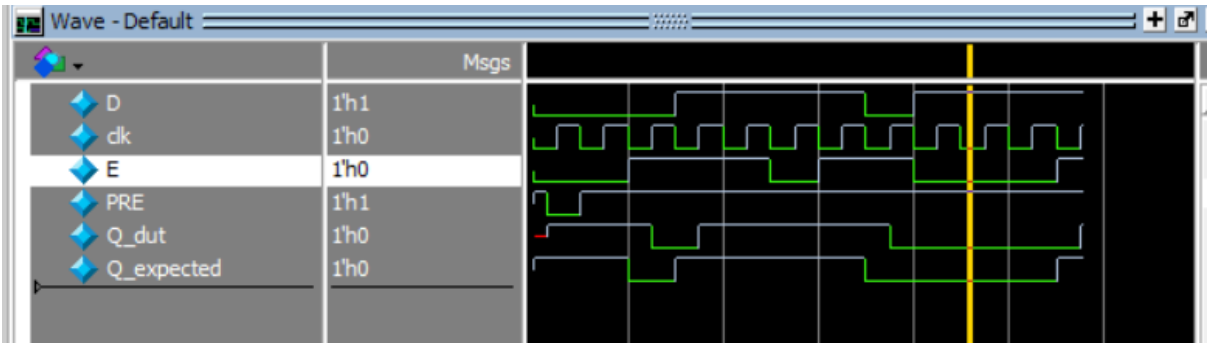
Omar Ahmed Abdelaty Mohammed



Submitted to: Eng. Kareem Waseem

Question 1

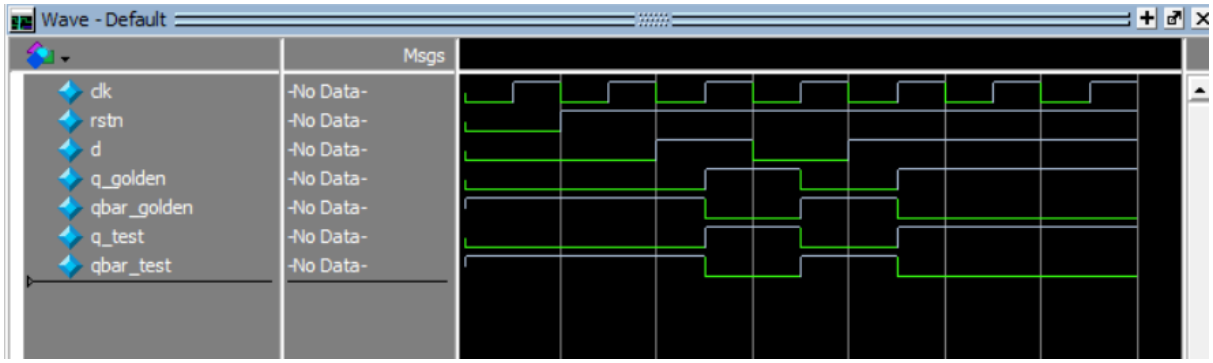
```
1  module Dff (
2      input D,clk,E,PRE,output reg Q
3  );
4
5  always @(posedge clk or negedge PRE) begin
6      if(!PRE)
7      Q<=1'b1;
8      else if(E)
9      Q<=D;
10 end
11
12 endmodule
```




```
1  vlib work
2  vlog Dff.v dff_tb.v
3  vsim -voptargs=+acc work.Dff_tb
4  add wave *
5  run -all
6  #quit -sim
```

```
module Dff_tb;

    reg D, clk, E, PRE;
    wire Q_dut;
    reg Q_expected;

    Dff DUT (
        .D(D),
        .clk(clk),
        .E(E),
        .PRE(PRE),
        .Q(Q_dut)
    );


    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
         D = 0; E = 0; PRE = 1;
        Q_expected = 1;


        #3 PRE = 0;
        @(negedge clk);
        PRE = 1;
        Q_expected = 1;


        repeat (10) begin
            @(negedge clk);
            D = $random;
            E = $random;


            if (!PRE) begin
                Q_expected = 1;
            end else if (E) begin
                Q_expected = D;
            end else begin
                Q_expected = Q_expected;
            end

            @(posedge clk);

            #1;

            if (Q_dut !== Q_expected) begin
                $display(" ERROR ");
                $stop;
            end
        end

        $display("Test passed");
        $stop;
    end
endmodule
```

Question 2




```
module d_t_ff(d, q, qbar, rstn, clk);
    parameter FF_TYPE = "DFF";
    input d, rstn, clk;
    output reg q, qbar;

    always @(posedge clk or negedge rstn) begin
        if (~rstn) begin
            q <= 1'b0;
            qbar <= 1'b1;
        end else begin
            if (FF_TYPE == "DFF") begin
                q <= d;
                qbar <= ~d;
            end else if (FF_TYPE == "TFF") begin
                if (d) begin
                    q <= ~q;
                    qbar <= ~qbar;
                end
            end
        end
    end
endmodule
```

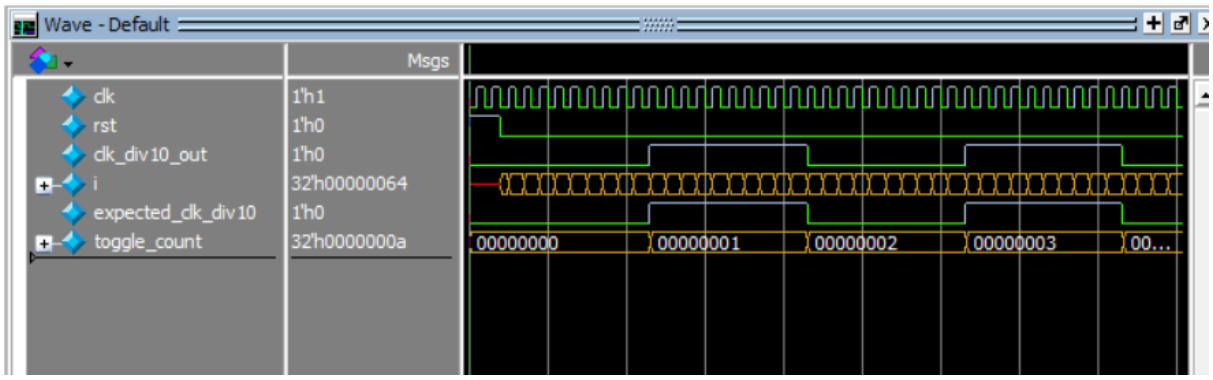
```
module tb_tff;

    reg clk, rstn, t;
    wire q_golden, qbar_golden;
    wire q_test, qbar_test;

    // Clock generation
    initial clk = 0;
    always #5 clk = ~clk;

    // Instantiate the TFF golden model (hardcoded FF_TYPE = "TFF")
    d_t_ff #(.FF_TYPE("TFF")) golden_tff (
        .clk(clk), .rstn(rstn), .d(t),
        .q(q_golden), .qbar(qbar_golden)
    );

    // Instantiate the parameterized design under test
    d_t_ff #(.FF_TYPE("TFF")) dut (
        .clk(clk), .rstn(rstn), .d(t),
        .q(q_test), .qbar(qbar_test)
    );

    initial begin
        $dumpfile("tff.vcd");
        $dumpvars(0, tb_tff);

        rstn = 0; t = 0;
        #12 rstn = 1;

        #10 t = 1;
        #10 t = 0;
        #10 t = 1;
        #10 t = 1;
        #10 t = 0;

         #30 $stop;
    end

endmodule

```

```
module tb_dff;

    reg clk, rstn, d;
    wire q_golden, qbar_golden;
    wire q_test, qbar_test;

    // Clock generation
    initial clk = 0;
    always #5 clk = ~clk;

    // Instantiate the DFF golden model (hardcoded FF_TYPE = "DFF")
    d_t_ff #(.FF_TYPE("DFF")) golden_dff (
        .clk(clk), .rstn(rstn), .d(d),
        .q(q_golden), .qbar(qbar_golden)
    );

    // Instantiate the parameterized design under test
    d_t_ff #(.FF_TYPE("DFF")) dut (
        .clk(clk), .rstn(rstn), .d(d),
        .q(q_test), .qbar(qbar_test)
    );

    initial begin
        $dumpfile("dff.vcd");
        $dumpvars(0, tb_dff);

        rstn = 0; d = 0;
        #12 rstn = 1;

        #10 d = 1;
        #10 d = 0;
        #10 d = 1;
        #10 d = 0;
        #10 d = 1;

        #30 $stop;
    end

endmodule
```

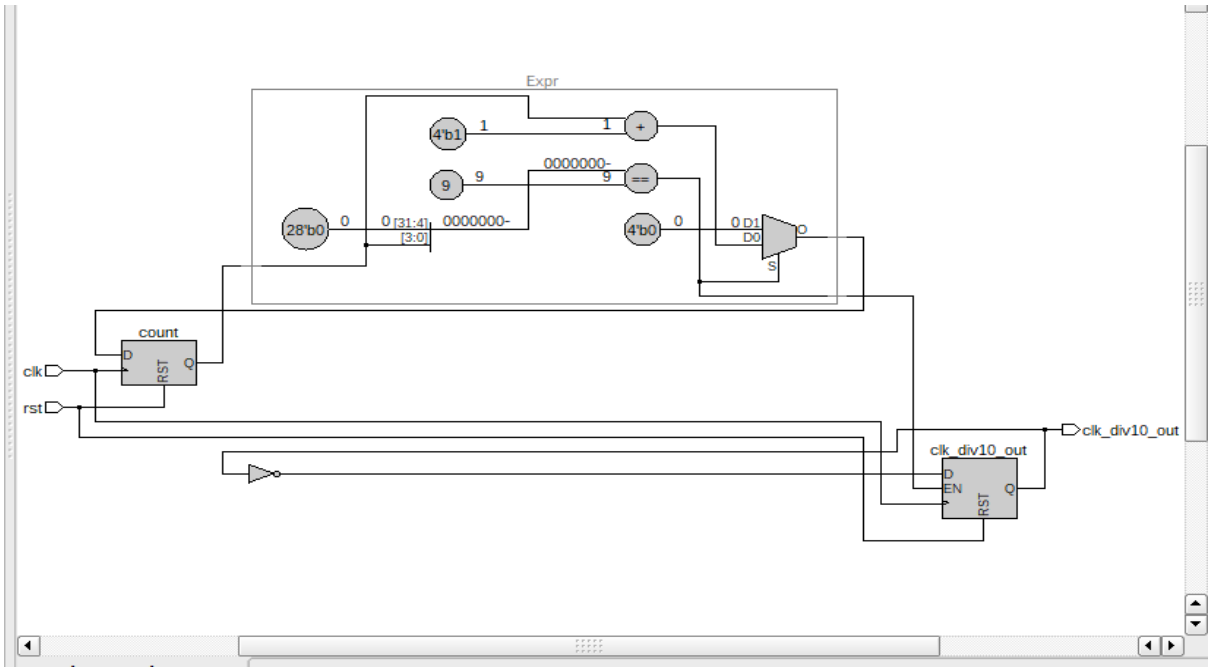
Severity	Status	Check	Alias	Message	Module	Category	State	Owner	STARC Reference
		multi_ports_in_single_line		Multiple ports are declared in one line. Module d_t_ff, ...	d_t_ff	Rtl Design Style	open	unassig...	3.5.6.3

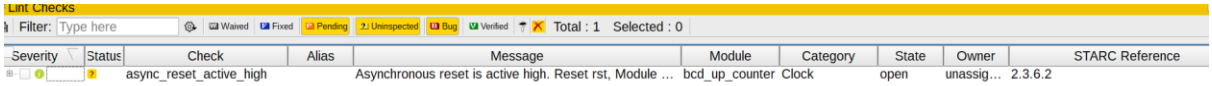
Question 3

```
module bcd_up_counter_tb;
    reg clk, rst;
    wire clk_div10_out;

    // Declare testbench variables
    integer i;
    reg expected_clk_div10;
    integer toggle_count;

    // Instantiate the Device Under Test (DUT)
    bcd_up_counter DUT (
        .clk(clk),
        .rst(rst),
        .clk_div10_out(clk_div10_out)
    );

    // Clock generation: 100MHz => 10ns period
    initial begin
        clk = 0;
        forever #5 clk = ~clk; // Toggle every 5ns → period = 10ns
    end

    // Test process
    initial begin
        $display("Starting BCD Up Counter Test...");

        // Initialize
        rst = 1;
        expected_clk_div10 = 0;
        toggle_count = 0;

        // Deassert reset after 20ns
        #20 rst = 0;

        // Wait and test for 100 clock cycles
        for (i = 0; i < 100; i = i + 1) begin
            @(posedge clk);

            // Every 10th rising edge, clk_div10_out should toggle
            if (i % 10 == 9) begin
                expected_clk_div10 = ~expected_clk_div10;
                #1; // Small delay to allow output to settle

                if (clk_div10_out !== expected_clk_div10) begin
                    $display("ERROR at cycle %0d: Expected clk_div10_out = %b, Got = %b", i+1, expected_clk_div10, clk_div10_out);
                    $stop;
                end else begin
                    toggle_count = toggle_count + 1;
                    $display("Correct toggle #%0d at cycle %0d", toggle_count, i+1);
                end
            end
        end

        $display("Test completed successfully! %0d toggles observed.", toggle_count);
        $stop;
    end
endmodule
```

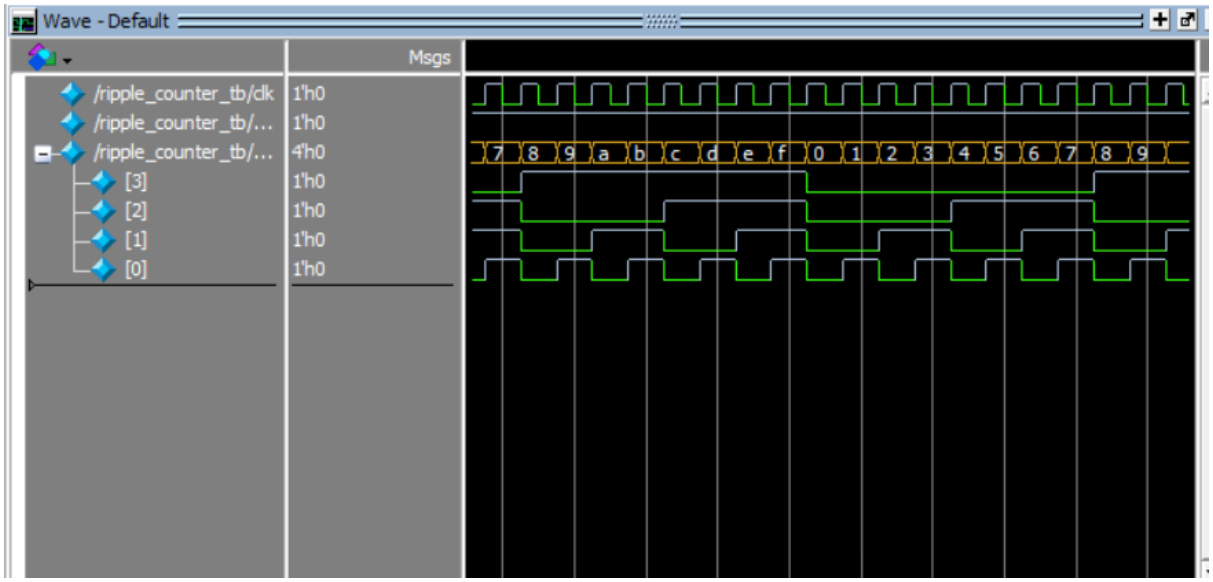


```verilog
module bcd_up_counter(
    input wire clk,
    input wire rst, // async active high
    output reg clk_div10_out
);

    reg [3:0] count;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            count <= 0;
            clk_div10_out <= 0;
        end else begin
            if (count == 9) begin
                count <= 0;
                clk_div10_out <= ~clk_div10_out; // Toggle output every 10 clocks
            end else begin
                count <= count + 1;
            end
        end
    end

endmodule
```





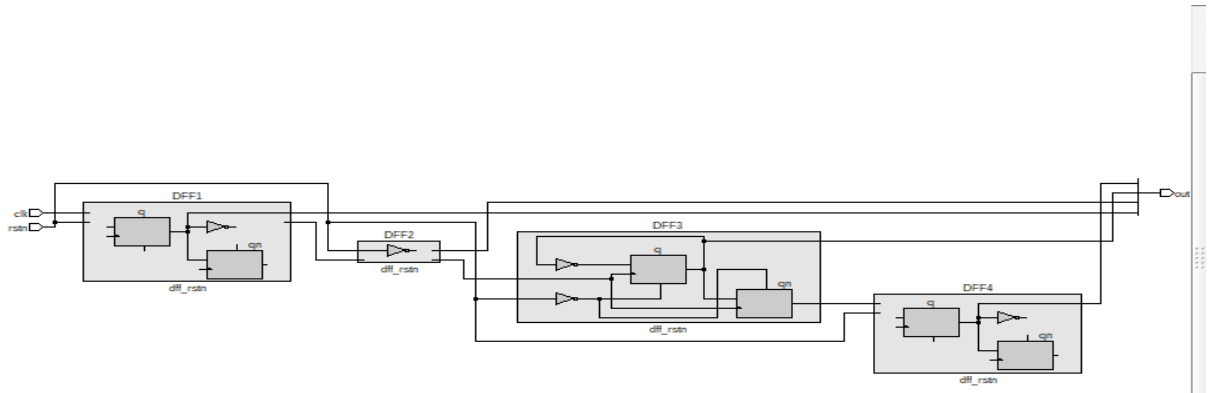
Question 4

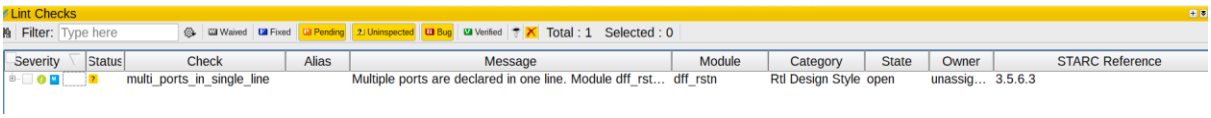



```
module ripple_counter (
    input clk,
    input rstn,
    output wire [3:0] out
);

    wire qn0, qn1, qn2;

    dff_rstn DFF1 (.clk(clk),   .rstn(rstn), .q(out[0]), .qn(qn0));
    dff_rstn DFF2 (.clk(qn0),   .rstn(rstn), .q(out[1]), .qn(qn1));
    dff_rstn DFF3 (.clk(qn1),   .rstn(rstn), .q(out[2]), .qn(qn2));
    dff_rstn DFF4 (.clk(qn2),   .rstn(rstn), .q(out[3]), .qn());

endmodule
```

```
module tb;
    reg clk, rstn;
    wire [3:0] out;

    ripple_counter UUT (
        .clk(clk),
        .rstn(rstn),
        .out(out)
    );

    // Clock generation
    initial begin
        clk = 0;
        forever #5 clk = ~clk;  // 10ns clock period
    end

    // Reset and simulation control
    initial begin
        rstn = 0;      // Start with reset active (low)
        #15 rstn = 1;  // Release reset after 15ns

        #200;          // Run for a while
        $stop;
    end

    // Monitor output
    initial begin
        $monitor("Time=%0t | out=%b", $time, out);
    end
endmodule
```

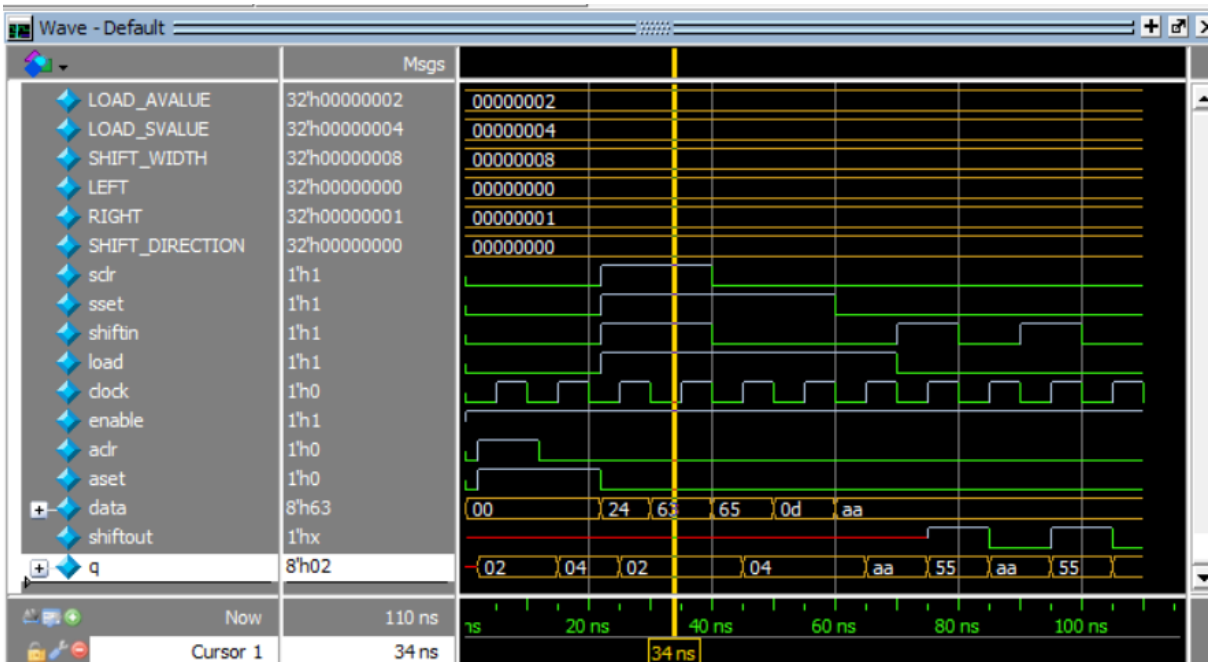
```verilog
module dff_rstn(q, qn, rstn, clk);
    input rstn, clk;
    output reg q, qn;

    always @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            q  <= 1'b0;
            qn <= 1'b1;
        end
        else begin
            q  <= ~q;
            qn <=  q;
        end
    end
endmodule
```





It is just a warning not error

Question 5

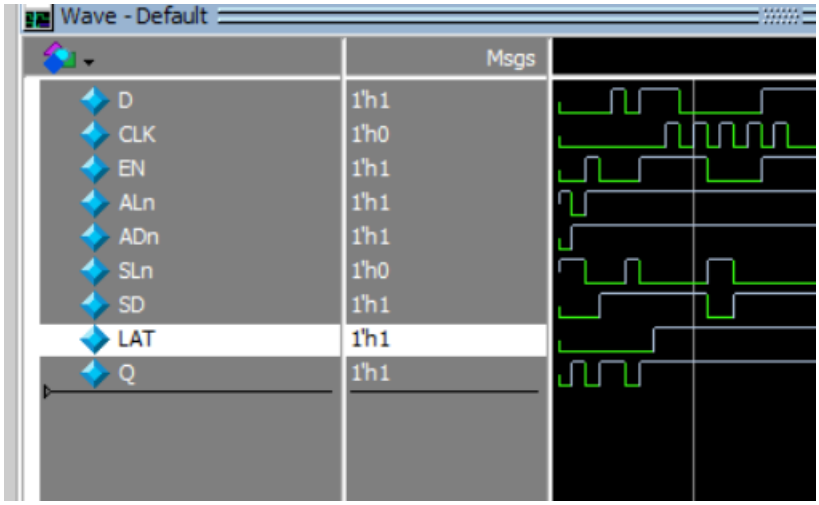

Wave - Default
Msgs
LOAD_AVALUE
LOAD_SVALUE
SHIFT_WIDTH
LEFT
RIGHT
SHIFT_DIRECTION
sclr
sset
shiftin
load
clock
enable
adr
aset
data
shiftout
q
32'h00000002
32'h00000004
32'h00000008
32'h00000000
32'h00000001
32'h00000000
1'h1
1'h1
1'h1
1'h1
1'h0
1'h1
1'h0
1'h0
8'h63
1'hx
8'h02
00000002
00000004
00000008
00000000
00000001
00000000
00
24
65
0d
aa
02
04
02
04
aa
55
aa
55
Now
110 ns
Cursor 1
34 ns
20 ns
40 ns
60 ns
80 ns
100 ns
34 ns

```
module param_shift_reg #(
    parameter integer LOAD_AVALUE = 1,
    parameter integer LOAD_SVALUE = 1,
    parameter integer SHIFT_DIRECTION = 0, // 0 = LEFT, 1 = RIGHT
    parameter integer SHIFT_WIDTH = 8
)(
    input  wire                  sclr,
    input  wire                  sset,
    input  wire                  shiftin,
    input  wire                  load,
    input  wire [SHIFT_WIDTH-1:0] data,
    input  wire                  clock,
    input  wire                  enable,
    input  wire                  aclr,
    input  wire                  aset,
    output reg                   shiftout,
    output reg [SHIFT_WIDTH-1:0] q
);

    always @(posedge clock or posedge aclr or posedge aset) begin
        if (aclr)
            q <= {SHIFT_WIDTH{1'b0}} + LOAD_AVALUE;
        else if (aset)
            q <= {SHIFT_WIDTH{1'b0}} + LOAD_SVALUE;
        else if (enable) begin
            if (sclr)
                q <= {SHIFT_WIDTH{1'b0}} + LOAD_AVALUE;
            else if (sset)
                q <= {SHIFT_WIDTH{1'b0}} + LOAD_SVALUE;
            else if (load)
                q <= data;
            else begin
                if (SHIFT_DIRECTION == 0) begin
                    shiftout <= q[SHIFT_WIDTH-1]; // shift left
                    q <= {q[SHIFT_WIDTH-2:0], shiftin};
                end else begin
                    shiftout <= q[0]; // shift right
                    q <= {shiftin, q[SHIFT_WIDTH-1:1]};
                end
            end
        end
    end
endmodule
```

```
module tb_param_shift_reg;

    // Shift direction as numeric constant
    localparam integer LOAD_AVALUE      = 2;
    localparam integer LOAD_SVALUE     = 4;
    localparam integer SHIFT_WIDTH     = 8;
    localparam integer LEFT            = 0;
    localparam integer RIGHT           = 1;
    localparam integer SHIFT_DIRECTION = LEFT;

    // DUT inputs
    reg  sclr, sset, shiftin, load, clock, enable, aclr, aset;
    reg  [SHIFT_WIDTH-1:0] data;

    // DUT outputs
    wire shiftout;
    wire [SHIFT_WIDTH-1:0] q;

    // Instantiate DUT
    param_shift_reg #(
        .LOAD_AVALUE(LOAD_AVALUE),
        .LOAD_SVALUE(LOAD_SVALUE),
        .SHIFT_DIRECTION(SHIFT_DIRECTION),
        .SHIFT_WIDTH(SHIFT_WIDTH)
    ) dut (
        .sclr(sclr), .sset(sset), .shiftin(shiftin), .load(load),
        .data(data), .clock(clock), .enable(enable),
        .aclr(aclr), .aset(aset), .shiftout(shiftout), .q(q)
    );

    // Clock generation
    always #5 clock = ~clock;

    // Stimulus
    initial begin
        $display("Starting test...");
        clock = 0;
        enable = 1;
        sclr = 0; sset = 0; aclr = 0; aset = 0;
        load = 0; shiftin = 0; data = 0;

        // 2.1 Async Clear & Set
        #2 aclr = 1; aset = 1; #10;
        if (q !== LOAD_AVALUE) $display("Error: Async clear failed");

        aclr = 0; aset = 0;

        // 2.2 Async Set
        aset = 1; #10;
        if (q !== LOAD_SVALUE) $display("Error: Async set failed");
        aset = 0;

        // 2.3 Sync Clear
        sclr = 1; sset = 1;
        repeat(2) begin
            data = $random;
            shiftin = $random;
            load = $random;
            @(negedge clock);
            if (q !== LOAD_AVALUE) $display("Error: Sync clear failed");
        end
        sclr = 0; sset = 0;

        // 2.4 Sync Set
        sset = 1;
        repeat(2) begin
            data = $random;
            shiftin = $random;
            load = $random;
            @(negedge clock);
            if (q !== LOAD_SVALUE) $display("Error: Sync set failed");
        end
        sset = 0;

        // 2.5 Load
        load = 1;
        data = 8'b10101010;
        @(negedge clock);
        if (q !== 8'b10101010) $display("Error: Load failed");
        load = 0;

        // 2.6 Shift (observe waveform manually)
        repeat(4) begin
            shiftin = $random;
            @(negedge clock);
        end

        $display("Test completed.");
        $stop;
    end
endmodule
```

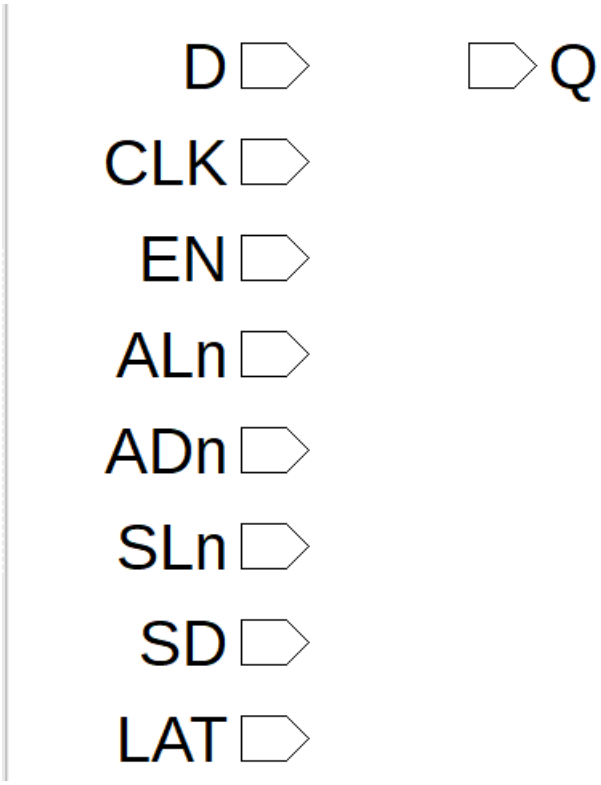
Severity	Status	Check	Alias	Message	Module	Category	State	Owner	STARC Reference
		async_reset_active_high		Asynchronous reset is active high. Reset aclr, Module…	param_shift_reg	Clock	open	unassig…	2.3.6.2
		async_reset_active_high		Asynchronous reset is active high. Reset aset, Modul…	param_shift_reg	Clock	open	unassig…	2.3.6.2

Question 6


Wave - Default
Msgs
D
CLK
EN
ALn
ADn
SLn
SD
LAT
Q
1'h1
1'h0
1'h1
1'h1
1'h1
1'h0
1'h1
1'h1
1'h1

```
module SLE (
    input wire D,
    input wire CLK,
    input wire EN,
    input wire ALn,
    input wire ADn,
    input wire SLn,
    input wire SD,
    input wire LAT,
    output reg Q
);

always @(*) begin
    if (!ALn)
        Q = ADn; // Asynchronous Load (ALn active low)
end

always @(posedge CLK or negedge ALn) begin
    if (!ALn)
        Q <= ADn; // Async override again in clocked block
    else if (LAT) begin // Flip-Flop mode
        if (EN) begin
            if (!SLn)
                Q <= SD; // Synchronous Load
            else
                Q <= D; // Normal data input
        end
    end
end

always @(*) begin
    if (!ALn)
        Q = ADn;
    else if (!LAT) begin // Latch mode
        if (!SLn)
            Q = SD;
        else
            Q = D;
    end
end

endmodule
```

```
module SLE_tb;
    reg D, CLK, EN, ALn, ADn, SLn, SD, LAT;
    wire Q;

    SLE uut (
        .D(D), .CLK(CLK), .EN(EN), .ALn(ALn),
        .ADn(ADn), .SLn(SLn), .SD(SD), .LAT(LAT), .Q(Q)
    );

    initial begin
        $dumpfile("sle.vcd");
        $dumpvars(0, SLE_tb);

        // Initialize inputs
        CLK = 0; D = 0; EN = 0; ALn = 1;
        ADn = 0; SLn = 1; SD = 0; LAT = 0;

        // Async load first
        #5 ALn = 0; ADn = 1;
        #5 ALn = 1;

        // Latch Mode Test
        LAT = 0;
        repeat (5) begin
            {D, EN, SLn, SD} = $random;
            #5;
        end

        // Flip-Flop Mode Test
        LAT = 1;
        repeat (5) begin
            {D, EN, SLn, SD} = $random;
            #5 CLK = 1;
            #5 CLK = 0;
        end

        #10 $stop;
    end
endmodule

```



Severity	Status	Check	Alias	Message	Module	Category	State	Owner	STA
⚠	?	inferred_blackbox		Module is inferred as blackbox. Module SLE, File D:/…	SLE	Rtl Design Style	open	unassig…	
⚠	?	reset_set_non_const_as…		Signal is assigned non-constant value under set/reset …	SLE	Reset	open	unassig…	

Warning not errors.